
Tutorial 4 Solution

Computer Security BCS2420

Part A

1. **Answer:** B. In key transport, one party picks the key and securely sends it; in key agreement, both parties' inputs determine the shared key.
 2. **Answer:** D. Symmetric key (it's not a typical TVP, but a long-term or session key, not a fresh parameter like a nonce/timestamp).
 3. **Answer:** A. The parties do not verify each other's public parameters.
 4. **Answer:** B. It involves sending a copy of a message back to the original sender.
 5. **Answer:** B. Actively forwarding protocol messages in real time between two unsuspecting parties.
 6. **Answer:** B. Offline attackers can replay $H(K)$ without knowing (K) .
 7. **Answer:** A. Explicit key authentication ensures key confirmation, while implicit only assures the key is shared by correct parties.
 8. **Answer:** B. It ensures that compromising a long-term key does not reveal *past* session keys.
 9. **Answer:** C. Keeping track of large prime factorization results is not typically a key-management concern.
 10. **Answer:** B. Using nonce-based challenges or timestamps.
-

Part B

1. Replays and Nonces

- A nonce is a random, one-time-use value included in a challenge to the other party. If the protocol ensures the responder's reply includes this nonce in a cryptographically secure manner (e.g., signed or hashed with a secret), an attacker cannot replay old messages successfully.
- If the recipient does *not* verify that the nonce was recently generated, the attacker might replay an old nonce-response pair that once worked, tricking the protocol into accepting it.

2. Authenticated vs. Unauthenticated DH

- **Unauthenticated** DH means an attacker can insert themselves (man-in-the-middle) and negotiate separate keys with each party. The parties do not confirm each other's identity or public parameters.
- **Authenticated** DH (using public-key signatures or shared secrets) ensures each ephemeral message is bound to a known identity. This prevents MITM by letting each side confirm the other's ephemeral exponent belongs to the correct entity.

3. Forward Secrecy Concepts

- Forward secrecy means that if long-term keys are compromised in the future, *past* session keys remain protected. This is crucial in TLS to ensure that recorded traffic cannot be decrypted retroactively.
- Without forward secrecy, an attacker who obtains the server's private key can decrypt all previously captured TLS sessions. That can expose old passwords, credit card data, etc.

4. Reflection vs. Relay

- **Reflection:** The attacker replays a challenge from the server back to the server itself (or from the client back to the client) by interleaving parallel runs.
- **Relay:** The attacker simply passes messages in real time between two legitimate parties who think they are talking to each other, but in fact the attacker stands in the middle.
- Example: Relay might fool a car's proximity sensor by

capturing signals from the key fob in the house and relaying them to the car outside.

5. Dictionary Attack Defenses

- Technical:

1. Use protocols that do not reveal direct hashes or short challenge responses.
2. Implement rate-limiting or lockout after a few failed attempts.

6. **Policy:** 1. Require stronger (longer) passwords, possibly passphrases. 2. Encourage multi-factor auth so even if the password is guessed, an additional factor is needed.

7. DH-EKE (Sketch)

- Basic flow:

1. A→B: A's ephemeral g^a encrypted under the password.
2. B→A: B's ephemeral g^b encrypted under the password.
3. Both derive a key from g^{ab} .

- Encrypting the ephemeral shares with the password hides them from offline attackers. Attackers can't reliably test guesses if they can't confirm which ephemeral exponent is correct.

8. Single Sign-On Threats

- **Risk 1:** If the SSO provider is compromised, all connected services are compromised.
- **Risk 2:** Token replay or forging SSO tokens (in OIDC or SAML), letting an attacker impersonate a user across multiple apps.
- **Mitigations:**
 - Harden the SSO IdP with strong security and monitoring.
 - Sign and validate tokens with short lifetimes; implement audience checks and signature verification at each service.

Part C

1. Number of Offline Guesses

- 10^{11} possible passwords, attacker tests 10^7 guesses/second.
- Time = $\frac{10^{11}}{10^7} = 10^4$ seconds = $10^4/3600 \approx 2.78$ hours.

2. Diffie-Hellman Security

- The discrete log problem for a 2048-bit prime is considered beyond 2^{60} operations in practice. Sub-exponential algorithms (e.g., number field sieve) still require huge resources. So performing 2^{60} direct tries is not nearly enough to break 2048-bit DH. The complexity is significantly higher than a naive approach might suggest.

3. Replay with Timestamps

- A message is valid within 2 minutes of its timestamp. The attacker can replay it successfully only within that 2-minute window. After that, the receiver rejects it as stale.

4. Diffie-Hellman EKE Overhead

- 2 exchanges, each side does encryption & decryption = 4 total operations if we count each direction separately (A encrypt, B decrypt, B encrypt, A decrypt).
- If each operation is 1ms, that's 4ms total. Or if we interpret "2 exchanges" as (A→B + B→A), that's 2 encryption + 2 decryption = 4ms overhead total.

5. SS0: Single Sign-On Attempt Time

- 5 SS0 checks × 50ms each = **250ms** total wait. (Unless the system caches the first SS0 token, but we ignore that.)